An Enhanced System on Chip-Based Sobel Edge Detector

Ahmed S. khalil
Electronics & Communications dept.
Faculty of Engineering Ain-Shams University
Cairo, Egypt
10333@eng.asu.edu.eg

Mohamed Shalaby
Computer Science dept.
Arab Academy for Science, Technology, Maritime Transport
Heliopolis, Cairo, Egypt
myousef73@hotmail.com

Emad Hegazi
Electronics & Communications dept.
Faculty of Engineering Ain-Shams University
Cairo, Egypt
emad.hegazi@eng.asu.edu.eg

2023 International Telecommunications Conference (ITC-Egypt) | 979-8-3503-2606-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/ITC-Egypt58155.2023.10206327

***Abstract*— Nowadays real-time image processing has become a great challenge because of the use of high-resolution camera sensors, which produce HD video streaming with high resolution and frame rate. As a result, processing this huge amount of data on serial processor became a big challenge. Programmable logic such as FPGA plays an important role in solving this problem by using parallel processing techniques like pipelining. The advances in the system on chip (SOC) technology allow researchers to develop complex heterogeneous systems that can be used in real-time image processing. In this paper, we present a hardware design implementation for accelerating Sobel edge detector by applying some optimization techniques such as line buffer and memory window technique, pipeline technique, unrolling for parallel processing technique, and loop-flatten technique. The experimental results of this design are presented and a comparison between this design and related work is discussed.**

Keywords*— *Sobel Filter, Edge Detection, Parallel Processing, FPGA, SOC, Real-time, Image Processing

I. INTRODUCTION

There is no doubt that real-time image processing becomes one of the most valuable point in research area due to the rapid development in the sensor resolution. It is a big challenge to achieve real-time image processing, due to the big amount of data to be processed in a short time, for example processing HD video of 1920 x1080p format, 60 Hz refresh rate, and 24 data bits per pixel, will take more than 373 million samples per second. Achieving this performance on a serial processor is hard as the sequence of the processing includes many sequential processing steps, namely, instruction fetch, instruction decode, instruction execute, memory operations and write back [1].

Edge detection is one of the most important pre-processing methods and it is based on the abrupt changes in the intensity [2]. Edges are usually used in image perception and analysis for finding the region of interest boundaries, which play an important role in many applications like feature extraction, motion detection, object tracking and data reduction. The edge property is calculated using the function behavior of the image in a neighborhood of that pixel. Operators which detect edges are represented by a collection of masks, each corresponding to a certain direction. There are two groups of gradient operators, the first group approximates the first derivative, examples of this group are Roberts, Prewitt, and Sobel, Robinson, and Kirsch operators. The second group is based on the zero-crossings of the image function second derivative, an example of this group is canny operator.

In the first group, the edges are detected in the image by calculating the gradient in the horizontal and vertical directions. The gradient gives the direction of maximal growth of the image function from black to white. The gradient magnitude and direction are continuous image functions and are calculated from Eq. (1) and Eq. (2).

The gradient magnitude:

$$|\text{grad } g(x,y)| = \sqrt{(\frac{\partial g}{\partial x})^2 + (\frac{\partial g}{\partial y})^2} \quad (1)$$

The gradient direction

$$\Psi = \arg(\frac{\partial g}{\partial x}, \frac{\partial g}{\partial y}) \quad (2)$$

In this work, we use Sobel edge detection which is an example of the first group.

Due to the limitations of the serial processing, researchers tackled the area of processing data in parallel to take the maximum performance such that more than one pixels are processed at the same time. Implementing parallel processing using hardware platform makes the processing faster.

Programmable logic like FPGAs (field programmable gate array) is one of the hardware accelerator processing resources due to its flexibility in implementation and it's a custom hardware solution. FPGA consists of a matrix of logic blocks that are connected by a switching network. Both the logic blocks and the switching network are reprogrammable allowing application specific hardware to be constructed, while at the same time maintaining the ability to change the functionality of the system with ease [1]. The two major image processing algorithms used in parallelism are temporal and spatial parallelism [3]. To achieve the maximum gain of these two forms FPGA, we need to use a mixture of these two forms to apply parallel processing by using FPGA parallel processing techniques such as concurrency, pipelining and priming.

Recently, System on Chip (SOC) is developed such that FPGA and processor are integrated on the same chip to avoid the

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:55:24 UTC from IEEE Xplore. Restrictions apply.

bottlenecks of connecting the FPGA to processor through the external media interface. Processing video data using SOC was improved as the delay of transferring data through external media does not exist anymore.

SOC is divided into two main discrete subsystems. The Processing System (PS) and the Programmable Logic (PL) [4]. The PS consists of many components including the Application Processing Unit, Memory controller, and other various peripheral controllers. The PL (FPGA) includes several dedicated ports and buses that tightly couple to the PS. In this work, we implement Sobel edge detection using Xilinx Zynq-7000 family. The (PS) subsystem is a dual-core ARM Cortex-A9 processor and the (PL) subsystem is Xilinx 7-series connected to each other using AXI protocol as shown in (Fig 1). We use the processor as coordinator to distribute tasks, and we use the FPGA to implement the Sobel edge detection algorithm. The implementation using FPGA reduces the processing time significantly.

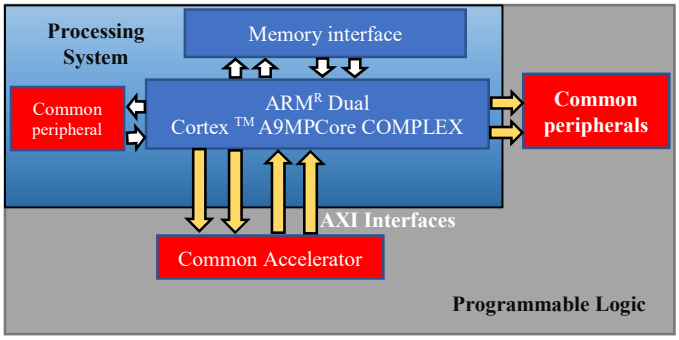

Figure. 1. Zynq-7000 based system architecture.

The rest of the paper is organized as follows. In section II we discuss the Sobel based edge detection in details. In section III, we present a related work. Our proposed work that includes the processing techniques required to accelerate the processing time is explained in section IV. The detailed hardware platform to stream HD video and apply Sobel edge detector is presented in section V. The experimental results and a comparison between the proposed work and the related work is presented in section VI. Finally, section VII concludes this work.

II. Sobel Filter Edge Detection Algorithm

One of the most common algorithms used in image edge detection is Sobel filter. This algorithm technique is based on a discrete differentiation operator, which approximates the computation of the gradient of the image intensity function [5]. The result at each pixel due to that algorithm is corresponding to the gradient orientation and magnitude which is based on convolving the image with a separable, small, and integer-valued filter in the vertical direction (G_x) and the horizontal direction (G_y). The 3x3 Sobel kernel is given by Eq. (3):

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \; G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} \quad (3)$$

Where the kernel G_x is sensitive to changes in the vertical direction and the kernel G_y is sensitive to changes in the horizontal direction. Gradient magnitude and direction are given by Eq. (4):

$$G = \sqrt{{G_x}^2 + {G_y}^2} \quad (4)$$

The approximation of the magnitude is calculated by Eq. (5):

$$|G| = |G_x| + |G_y| \quad (5)$$

The orientation angle of the edge giving rise to the spatial gradient is given by Eq. (6):

$$\Theta = \text{atan}\left(\frac{G_y}{G_x}\right) \quad (6)$$

III. Related Work

In this section, we introduce the conventional approach presented in [12], where the authors proposed a hardware implementation of Sobel filter. They used Digilent ZYBO Zynq ZC7010 board with 150MHZ clock. They used the frame buffer technique to receive data and buffer it into the FPGA. The frame buffer is big enough to accommodate the 1920x1080 video frame. Using frame buffer technique, the entire input video frame is stored on FPGA directly and there is no need to access DDRAM. After that, convolutions over 2,073,600 pixels is conducted. The conventional approach uses 816 flip-flop (FF), 1203 look-up table (LUTs) and 5 block ram (BRAM). Here, frame buffering and convolution are performed sequentially, and thus this design consumes a huge amount of resources. In section IV, we improve this work by proposing a design that uses the shift line buffer technique and memory window rather than frame buffer technique. This design allows both buffering and convolution of one frame to be performed in parallel, and hence we save resources (the number of clock cycles and FPGA components). Moreover, we use pipelining such that FPGA can access 60 frames from DDRAM simultaneously.

IV. The Proposed Work

Image frames when transferred are scanned using raster-scan as shown in (Fig 2) [6].

Figure 2: Raster Scan.

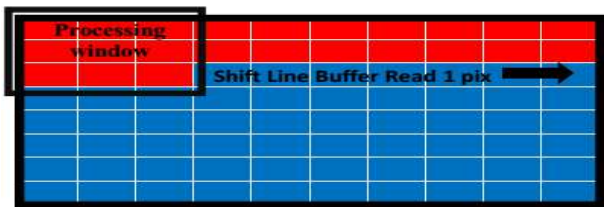

Figure 3: Direct stream using LP and MW.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:55:24 UTC from IEEE Xplore. Restrictions apply.

Transferring the data from CPU or system memory to FPGA and vice versa must be performed in the standard raster-scan shown in (Fig 2). This the reason why we build our work on streaming the input video pixels in sequential order. One method to implement Sobel edge detection using a 3x3 convolution kernel is streaming the 9 pixels that forms this kernel into FPGA then applying convolution to them. However, this method is slow because DDRAM is accessed nine times sequentially for each kernel. For example, if the frame size is 1080x1920 the total number of pixels will be 2073600 pixels and then the total number of memory access is $9 \times 2073600 = 18662400$. This problem was solved by some methods such as frame buffer method (which was discussed in section III), and shifting line buffer and memory window. In this work, we use shifting line buffer (LP) and memory window (MW) as shown in (Fig 3). The input video pixels are streamed one at a time in the FPGA line buffer, each time the streamed pixels are shifted. When the line buffer has enough pixels such that convolution can be applied, both streaming and convolution are performed simultaneously. In this case, each pixel is read only once and this results in 9x less processing time [7]. By implementing the line buffer, we can simply replace the memory access with the buffer access. To access all the convolution kernel pixels in one clock, we use memory window. The memory window is a storage element implemented using the FF resources from the FPGA fabric. Each register in this memory can be accessed independently and simultaneously [8]. Another problem that we addressed is handling the 60 frames/s simultaneously. To understand this problem, consider a video stream with frame rate equals to 60 frame/s and frame size 19020x1080. It takes $60 \times 1920 \times 1080 = 124$ million clock cycles/s to finish the processing of Sobel edge detection. Therefore, we have used pipelining so that 60 frames can be processed concurrently rather than waiting for the previous frame processing to be completed. The initial interval of pipeline equal one clock and hence the number of clock cycles/s required to handle the video stream mentioned above is $60 + 1920 \times 1080 \approx 2$ million clock cycles/s. We use loop-flatten technique which allows nested loops to be collapsed into a single loop to remove the loop transition overhead. To increase hardware parallelism, the convolution kernel window loop is unrolled and hence creating multiple independent operations rather than single operation. The final optimization step, is inlining functions so that the function hierarchy is removed and more than one function can be executed simultaneously, in this case we use less number of logic elements that leads to logic optimization across function boundaries and therefore latency is improved as the function call overhead is reduced.

V. HARDWARE PLATFORM FOR HD VIDEO STREAMING AND SOBEL EDGE DETECTOR

In our implementation, we have used SDSOC software development environment targeting the Zynq SOC. This tool can serve as an alternative solution to the HDL IPs coding and behavioral simulation in traditional FPGA design flow. Designers start coding using a high-level language, for example, C/C++ or SystemC with proper restructurings, then they verify and test their algorithms using software libraries and tools, i.e., OpenCV, OpenGL and Eclipse-based integrated development environment (IDE). This tool also comes with a powerful performance estimation tool to make the user take decision to his scenarios before implementation as shown in (Fig 4) [9].

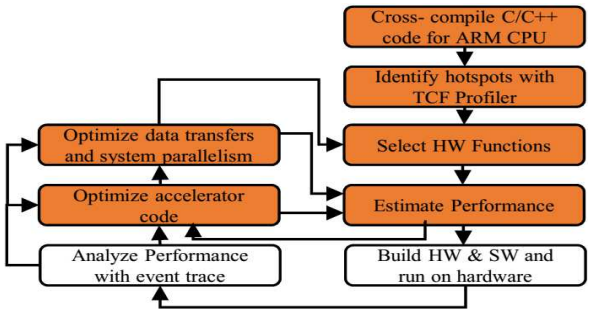

Figure 4: System optimization steps.

From Eq. 4, 5, and 6, we can see that if we need to compute the edge detection for HD video 1920 x1080p with 4:2:2 YCbCr video format, we need to move a kernel box of size 3x3 at all the pixels on the frame. This takes a huge amount of cycles to process each frame, also we need to store the total frame that is being processed now in the memory and this needs a big memory. Due to this difficulty, we have used SOC Zynq to combine real-time operating system in our case (standalone RTOS) and FPGA accelerating technique mentioned in section IV to addresses these difficulties as shown in (Fig 5).

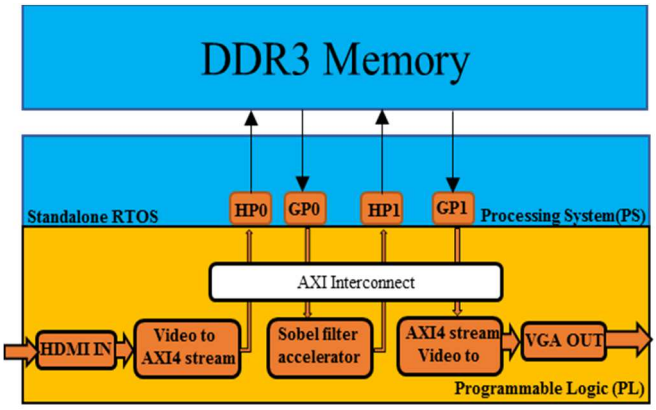

Figure. 5. Sobel edge detection accelerator block diagram.

The flow of the system design is described in this section as follows. The input video is streamed from HDMI camera HERO3 with resolution 1080p. The streaming video is directly connected to the programmable logic (PL) using custom IPs to read the data and convert it to YUV 4:2:2 video format that contains 16 bits of data which is more straightforward. The Y channel of the YUV 4:2:2 (that provides most information that computer vision needs for video processing) is connected to "video in to AXI4-Stream" IP core. "video in to AXI4-Stream" is an IP core that works as a bridge between the video input and the video processing cores and is controlled by video timing controller (VTC) which is used to specify timing of the input signals to the "video in to AXI4-Stream" IP core [10]. After that, the data is passed to video direct memory access (VDMA) that provides a high-bandwidth direct memory access between memory and AXI4-Stream through the video type target peripherals including peripherals which support the AXI4-Stream Video protocol [11]. VDMA is responsible for transmitting the data between the FPGA and the DDR3 memory

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:55:24 UTC from IEEE Xplore. Restrictions apply.

that makes the data accessible by both the programmable logic (PL) and processing system (PS) which is efficient for high throughput. In the next step, the data is transmitted to the Sobel accelerator which first converts the image to grayscale to reduce the amount of data to be processed and increase throughput then applies Sobel algorithm. In this step, we use FPGA optimization techniques to accelerate the processing by using line buffer and memory window to read the streaming data, inline the function for logic optimization, apply pipeline and unrolling for parallel processing, and finally, apply loop-flatten to reduce the fractions of clock wasted between loop transition. The last step is the data passed through AXI4-Stream to video out and accustom IPs to be displayed on the screen using VGA format. All of these IP cores is connected to each other using Vivado IP integrator.

VI. SYSTEM IMPLEMENTATION AND EXPERIMENTAL RESULTS

In this section, we present the system implementation that consists of HD camera sensor HERO3 with resolution 1080x1920 at 60 frame/s, Digilent ZYBO Zynq ZC7010, and Samsung LED monitor as shown in (Fig 6). The input (unprocessed) video is streamed through HDMI and the output (filtered) frame is displayed through VGA on LED monitor. We show the experimental results of this system, then we compare these results with the results of the related work discussed in section III. Our system is working with frequency 117 MHZ and Sobel filter with 3x3 convolution kernel.

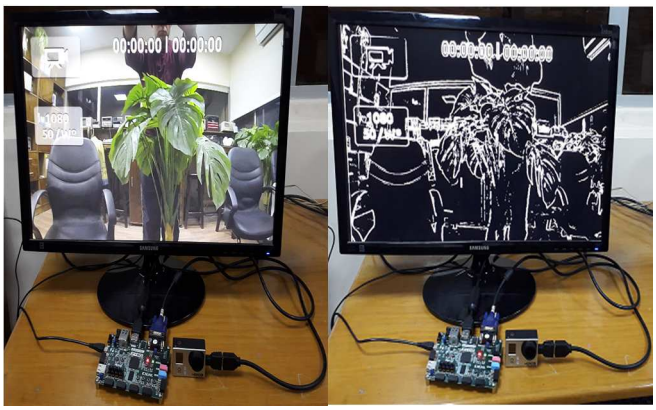

Figure. 6. Experimental result of implemented Sobel Edge detection For HD video input

A. Experimental results

The analysis of our system is based on different perspectives. First, we analyze our system according to resources utilization, second, we analyze it regarding hardware acceleration, and finally we analyze it regarding the number of CPU instructions per cycle. Table 1 shows the resource utilization of DSP, BRAM, LUT, and FF as well as the number of clock cycles used by our system without applying any optimization techniques. Table 2 shows the same results after applying the shift line buffer and the memory window techniques and inline these functions. It is clear that the number of used clock cycles is improved nine times as now, DDRAM is accessed nine times less for each kernel. Table 3 also shows the same results after applying the pipeline, unrolling, and loop flatten. These techniques accelerate the system with about 62 times more than the system takes without their application.

TABLE I. Sobel Edge detection resources utilization without optimization.

Performance estimates for 'SobelFilter' function

HW accelerated (Estimated cycles)	6493103766

Resource utilization estimates for hardware accelerators

Resource	Used	Total	% Utilization
DSP	5	80	6.25
BRAM	5	60	8.33
LUT	700	17600	3.98
FF	656	35200	1.86

TABLE II. Sobel Edge detection resources utilization with line buffer and memory window.

Performance estimates for 'SobelFilter' function

HW accelerated (Estimated cycles)	721455974

Resource utilization estimates for hardware accelerators

Resource	Used	Total	% Utilization
DSP	5	80	6.25
BRAM	3	60	5
LUT	541	17600	3.07
FF	517	35200	1.47

TABLE III. Sobel Edge detection resources utilization with line buffer and memory window and pipeline.

Performance estimates for 'SobelFilter' function

HW accelerated (Estimated cycles)	11640189

Resource utilization estimates for hardware accelerators

Resource	Used	Total	% Utilization
DSP	3	80	3.75
BRAM	3	60	5
LUT	508	17600	2.89
FF	540	35200	1.53

We design our system such that all image processing tasks are executed by the FPGA, and the CPU is only working as a coordinator of these tasks. This design reduces the number of CPU instructions per cycle significantly. (Figs 7, 8) use the performance analysis tool of SDSOC software to show the difference in the number of CPU instructions per cycle when running Sobel filter on CPU (the right side of the curve starting from time 0:01:05) and running it on FPGA (the left side of the curve ending at time 0:01:05).

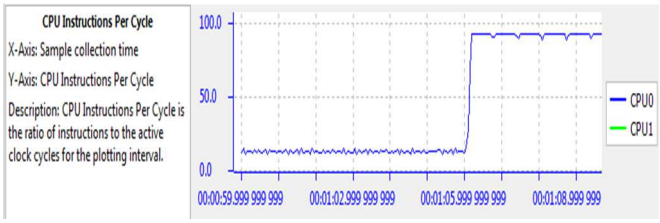

Figure 7: ARM processor instruction per cycle.

PS Performance Graphs | PS Performance Counters

00:00:00.000 000 000-00:01:02.669 999 999	CPU0	CPU1
CPU Utilization(%)	1.58	0.00
CPU Instructions Per Cycle	13.6	0.00
L1 Data Cache Miss Rate(%)	0.00	0.00
L1 Data Cache Access per ms	32.8k	0.00
CPU Write Stall Cycles Per Instruction	0.00	0.00
CPU Read Stall Cycles Per Instruction	0.00	0.00

PS Performance Graphs | PS Performance Counters

00:00:00.000 000 000-00:01:00.529 999 999	CPU0	CPU1
CPU Utilization(%)	1.58	0.00
CPU Instructions Per Cycle	93.2	0.00
L1 Data Cache Miss Rate(%)	0.16	0.00
L1 Data Cache Access per ms	195.9k	0.00
CPU Write Stall Cycles Per Instruction	0.00	0.00
CPU Read Stall Cycles Per Instruction	0.00	0.00

Figure. 8. ARM processor performance analysis with Sobel accelerator and without Sobel accelerator.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:55:24 UTC from IEEE Xplore. Restrictions apply.

B. *The comparison between proposed system and related work*

In this section, we present a comparison between our proposed system and the related work discussed in section III. We compare the two systems according to the utilization of FF, LUTs, BRAM, and clock. Table IV shows that the proposed system outperforms the related work. The proposed system is using line buffer and memory window technique rather than the frame buffer technique used in the related work, therefore the number of FF is reduced by 33.7% and the number of BRAM by 40 %. The proposed system also uses the pipeline, unrolling for parallel processing, and loop-flatten techniques and hence the number of LUTs is reduced by 57.77 % and the clock frequency reduced by 22 %.

TABLE IV. Resource comparison.

Resource	***Available***	***Related work used***	***Our work used***	Enhancement percentage %
FF	35200	816	540	33.7
LUTs	17600	1203	508	57.77
BRAM	60	5	3	40
clock	450MHZ	150	117	22

VII. CONCLUSION

In this paper, we present a hardware system for accelerating Sobel edge detector. The acceleration is based on designing the hardware such that all image processing tasks are executed on the FPGA and the coordination tasks are executed on the CPU. Also, the hardware design considers the application of some enhancement techniques, such as line buffer and memory window, pipeline, unrolling for parallel processing, and loop-flatten.

The HD video with resolution equals to 1080p and 60 frame/s is streamed from HD camera sensor HERO3 to the proposed system through HDMI input. The accelerated Sobel edge detector that runs on the Digilent ZYBO Zynq ZC7010, is applied to this video stream, and then the filtered frame is displayed on the monitor through VGA port.

The experimental results show that the proposed system, when compared with the system presented in the related work, decreases the resource utilization with 33.7% regarding FF, 57.77% regarding LUT, 40% regarding BRAM, and 22% regarding clock cycles.

As a future work, we will apply this accelerated design with more complicated image processing algorithms such as motion detection and object recognition.

REFERENCES

[1] Xilinx Inc. UG998, “Introduction to FPGA Design with Vivado High-Level Synthesis” , v.1.0, January 2019.

[2] Milan Sonka, Vaclav Hlavac, Roger Boyle, Image processing analysis and machine vision, 4th ed., Cenage learning, 2014, pp. 1-133-59360-7

[3] Downton, A. and Crookes, D., “Parallel Architectures for Image Processing” Electronics & Communication Engineering Journal, vol. 10, pp. 139-151, Jun, 1998.

[4] Digilent Inc, “ZYBO Reference Manual” , rev.B, pp. 502-279, February 2017.

[5] Samta Gupta, Susmita Ghosh Mazumdar, “Sobel Edge Detection Algorithm” International Journal of Computer Science and Management Research, vol. 2, pp. 2278-733, February, 2013.

[6] Xilinx Inc. UG902, “Vivado Design Suite User Guide High-Level Synthesis” , v. 2020.1, May 2021.

[7] Xilinx Inc. UG1235, “SDSoC Environment optimization Guide” , v. 2017.2, August 2017.

[8] C. T. Johnston, K. T. Gribbon, D. G. Bailey, “Implementing Image Processing Algorithms on FPGAs”,2016.

[9] Xilinx Inc. UG1146, “SDSoC Environment Platform Development Guide” , v. 2017.4, January , 2018.

[10] Xilinx Inc. PG043, “Video In to AXI4-Stream” , v. 5, November 2022.

[11] Xilinx Inc. PG020, “AXI Video Direct Memory Access” , v. 6.3, November 2022.

[12] Abdelkader BEN AMARA, Edwige PISSALOUX, Mohamed ATRI, “Sobel Edge Detection System Design and Integration on an FPGA Based HD Video Streaming Architecture” 11th International Design & Test Symposium, 2016.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:55:24 UTC from IEEE Xplore. Restrictions apply.